



RGB-D camera



LiDAR sensor



Stereo vision
system



Event-based
camera



Monocular camera



Infrared camera

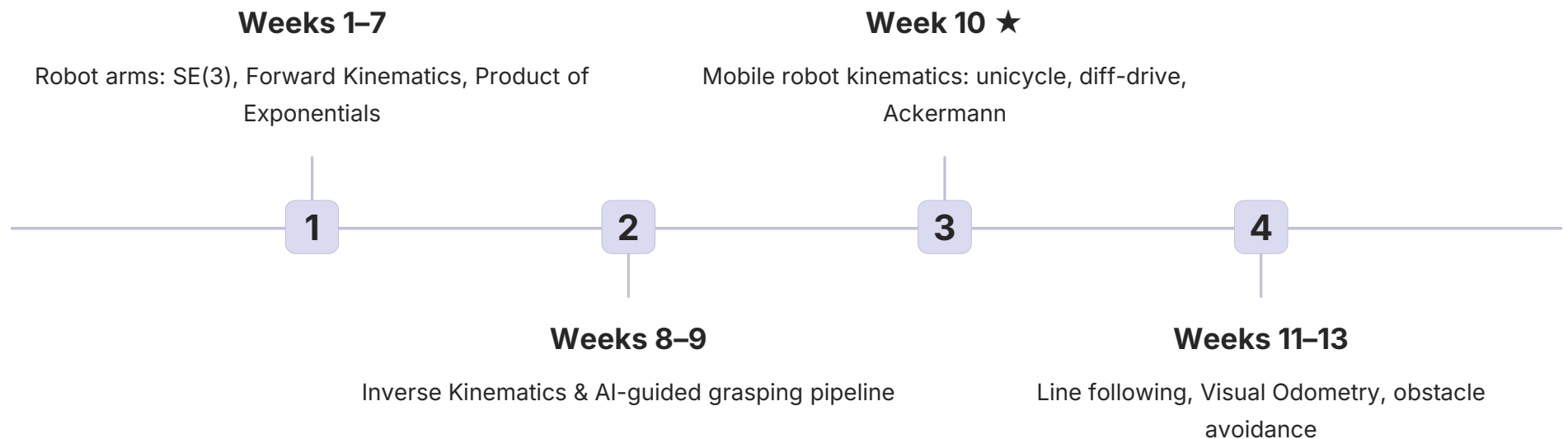
Robotics & Computer Vision

Week 10: Mobile Robot Kinematics

Unicycle, Differential Drive & Ackermann Steering

Lê Đình Phong, Ph.D

This week marks a major transition: from robot arms to mobile robots. Students now see that a robot can not only **grasp objects** but also **navigate autonomously** through an environment.



What You Will Master This Week

1

Unicycle Model

Understand the (x, y, θ) state with control inputs (v, ω) — the fundamental abstraction for most mobile robots.

2

Differential Drive

Derive the diff-drive model from unicycle and map between (v, ω) and wheel speeds (v_L, v_R) .

3

Ackermann Steering

Understand car-like kinematics for the Jet Racer and map steering/throttle SDK commands to (v, δ) .

Mobile Robot Types in RCV

The RCV course works with two hardware platforms, each representing a different kinematic family.



Three Families of Mobile Robots



Differential Drive

Two independently driven wheels. Examples: **Turtlebot**, **Pioneer**. Can spin in place — very maneuverable in tight spaces.



Car-like (Ackermann)

Rear-wheel drive, front steering. Example: **Jet Racer**. Finite turning radius — suited for high-speed navigation.



Omnidirectional

Mecanum, omni, skid-steer robots. Can move in any direction instantly. Covered conceptually in this course, not in depth.

A Universal Abstraction

State & Inputs

State vector: $\mathbf{q} = (x, y, \theta)$

Position in the plane + heading angle θ

Control inputs:

v — linear (forward) velocity

ω — angular (rotational) velocity

Kinematic Equations

$$\dot{x} = v \cos \theta$$

$$\dot{y} = v \sin \theta$$

$$\dot{\theta} = \omega$$

These three ODEs fully describe unicycle motion. Simple, yet powerful enough to model Turtlebot, Pioneer, and many other platforms.

Geometric Meaning of the Unicycle Model

The unicycle model captures one essential constraint: **the robot always moves in the direction it is facing**. It cannot slide sideways.

Straight Line

$$v > 0, \omega = 0$$

Robot moves forward along current heading θ . Infinite turning radius.

Spin in Place

$$v = 0, \omega \neq 0$$

Robot rotates about its center without any translational motion.

Circular Arc

$$v > 0, \omega > 0$$

Robot traces a circular arc. Radius $R = v / \omega$.

 Understanding these three cases is the key to intuition for all mobile robot motion planning.

The Nonholonomic Constraint

What It Means

The robot **cannot move instantaneously sideways** — it is constrained to move along its heading direction at every instant. This is called a **nonholonomic constraint**.

Mathematical Form

$$-\sin \theta \dot{x} + \cos \theta \dot{y} = 0$$

The lateral velocity component is always zero. This constraint is *non-integrable* — it restricts velocities, not positions.

Why It Matters

Unlike robot arms (holonomic), mobile robots **cannot reach arbitrary configurations instantaneously**. You must plan a path that respects this constraint — just like parallel parking a car.

 This makes mobile robot control fundamentally different from arm control.

Integrating Unicycle Trajectories

When v and ω are constant, the unicycle equations can be solved analytically:

Constant v, ω (circular arc)

$$\theta(t) = \theta_0 + \omega t$$

$$x(t) = x_0 + \frac{v}{\omega} (\sin \theta(t) - \sin \theta_0)$$

$$y(t) = y_0 - \frac{v}{\omega} (\cos \theta(t) - \cos \theta_0)$$

Special case: $\omega \rightarrow 0$ (straight line)

$$x(t) = x_0 + vt \cos \theta_0$$

$$y(t) = y_0 + vt \sin \theta_0$$

As $\omega \rightarrow 0$, the circular arc degenerates to a straight line. This is the limiting case used in Euler integration when ω is very small.



From Unicycle to Differential Drive

A differential drive robot has two independently actuated wheels. The **track width** W (distance between wheels) connects wheel speeds to the unicycle model.

Differential Drive Kinematic Mapping

Forward Kinematics (FK)

From wheel speeds to unicycle inputs:

$$v = \frac{v_R + v_L}{2}$$

$$\omega = \frac{v_R - v_L}{W}$$

W = track width (distance between wheels)

Inverse Kinematics (IK)

From unicycle inputs to wheel speeds:

$$v_R = v + \omega \frac{W}{2}$$

$$v_L = v - \omega \frac{W}{2}$$

This "IK" for diff-drive mirrors the arm IK from Week 8 — converting a high-level command into actuator commands.

 This mapping will be reused for Turtlebot kinematics and control throughout Weeks 10–13.

Typical Motion Modes of Diff-Drive

1

Straight Line

$$v_R = v_L$$

$\omega = 0$, infinite turning radius. Robot drives straight ahead.

2

Spin in Place

$$v_R = -v_L$$

$R = 0$. Robot rotates about its own center. Unique to diff-drive!

3

Pivot on One Wheel

$$v_R = 0 \text{ or } v_L = 0$$

Robot rotates around the stationary wheel.

4

Circular Arc

$$v_R \neq v_L, \text{ same sign}$$

Robot follows a curved arc with $R = v / \omega$.

Modeling Turtlebot with the Unicycle



Two-Level Architecture

At the **high level**, Turtlebot accepts **linear.x** (v) and **angular.z** (ω) commands — exactly the unicycle inputs.

The **low-level driver** internally converts (v, ω) to the actual left/right wheel speeds (v_L, v_R) using the IK formula.

This abstraction lets you design planners and controllers entirely in unicycle space, without worrying about hardware details.

✓ Use unicycle for planning → driver handles wheel mapping automatically.



Car-Like Robots & Ackermann Steering

The **Jet Racer** is a car-like robot: rear-wheel drive, front-wheel steering. Its kinematics differ fundamentally from diff-drive because the robot cannot spin in place.

Ackermann Steering Geometry

State & Inputs

State: (x, y, θ) — same as unicycle

Inputs:

v — longitudinal (forward) speed

δ — front wheel steering angle

L — wheelbase (front to rear axle)

Goal of Ackermann Geometry

Both front wheels rotate around the **same instantaneous center** to avoid tire slip — inner wheel turns more than outer wheel.

Kinematic Bicycle Model

$$\dot{x} = v \cos \theta$$

$$\dot{y} = v \sin \theta$$

$$\dot{\theta} = \frac{v}{L} \tan \delta$$

Similar to unicycle, but ω is *not independent* — it is coupled to both v and δ . This coupling makes control more complex than diff-drive.

Unicycle vs. Ackermann: Key Differences

Unicycle / Diff-Drive

v and ω are fully independent inputs. Set any combination freely.
Can spin in place ($\omega, v=0$). Easier to control and plan paths for.

Ackermann / Car-like

Angular rate $\omega = (v/L) \tan \delta$ — coupled to both speed and steering angle. Minimum turning radius. Control is more complex but suits high-speed operation.

❗ RCV uses both Turtlebot and Jet Racer so students experience both kinematic families firsthand.

Jet Racer in the RCV Program



SDK Interface

The Jet Racer Python SDK accepts two normalized commands:

- **steering** $\in [-1, 1]$: left to right
- **throttle** $\in [-1, 1]$: reverse to full forward

Mapping to Kinematic Model

$$v = \text{throttle} \times v_{\max}$$

$$\delta = \text{steering} \times \delta_{\max}$$

This lets you use the Ackermann simulation model **before** deploying on real hardware — greatly reducing risk during testing.

Week 10 Lab Structure

The lab is divided into three parts, progressively building from the basic unicycle model to Jet Racer simulation and path-following preparation.

1**Part A**

Unicycle & Diff-Drive kinematics —
simulate trajectories in Python

2**Part B**

Ackermann & Jet Racer — simulate car-like
trajectories with SDK mapping

3**Part C**

Path-following intro — connect error to (v, ω) / steering as prep for Week 11

Lab 10.1 — Writing `unicycle_step`

This is the fundamental building block for all mobile robot trajectory simulation. Implement a single Euler integration step for the unicycle model.

```
def unicycle_step(x, y, th, v, w, dt):  
    x_new = x + v * np.cos(th) * dt  
    y_new = y + v * np.sin(th) * dt  
    th_new = th + w * dt  
    return x_new, y_new, th_new
```

Inputs

- Current state: (x, y, θ)
- Control commands: v (m/s), ω (rad/s)
- Time step: dt (seconds)

Output

- New state $(x_{\text{new}}, y_{\text{new}}, \theta_{\text{new}})$
- Uses first-order Euler integration
- Foundation for Labs 10.2–10.5

Lab 10.2 — Simulating Basic Unicycle Trajectories

Use `unicycle_step` in a loop and plot the resulting (x, y) trajectory. Explore three fundamental cases:

Case 1: Straight Line

$v = \text{const}, \omega = 0$

Robot moves in a straight line. Verify $x(t) = x_0 + vt \cos\theta_0$.

Case 2: Circle

$v = \text{const}, \omega = \text{const} \neq 0$

Robot traces a circle of radius $R = v/\omega$. Verify analytically.

Case 3: Free Trajectory

$v(t), \omega(t)$ time-varying

Try sinusoidal $\omega(t)$ for a slalom path, or step changes for a zig-zag.

Lab 10.2 — Simulation Loop Code

Here is the simulation scaffold. Change `v` and `w` to explore different trajectory shapes:

```
dt      = 0.05      # time step (seconds)
steps   = 400       # total simulation steps
x, y, th = 0.0, 0.0, 0.0
traj = []

for k in range(steps):
    t = k * dt
    v = 0.2          # try changing this
    w = 0.5          # try 0, 0.5, -0.3, sin(t)
    x, y, th = unicycle_step(x, y, th, v, w, dt)
    traj.append((x, y))

# Plot trajectory
import matplotlib.pyplot as plt
xs, ys = zip(*traj)
plt.plot(xs, ys); plt.axis('equal'); plt.show()
```

❏ Always use `plt.axis('equal')` so circles appear as circles, not ellipses.

Lab 10.3 — Implementing `diff_drive_step`

Differential drive is simply a **physical realization** of the unicycle model. Implement it by converting wheel speeds to unicycle inputs and calling `unicycle_step`:

```
def diff_drive_step(x, y, th, v_L, v_R, W, dt):  
    v = (v_R + v_L) / 2.0  
    w = (v_R - v_L) / W  
    return unicycle_step(x, y, th, v, w, dt)
```

Key Insight

`diff_drive_step` is a thin wrapper around `unicycle_step`. This reveals that diff-drive *is* a unicycle — just expressed in wheel-speed coordinates.

Parameter

W = track width in meters (e.g. 0.3 m for Turtlebot). This is the only hardware-specific parameter needed.

Lab 10.4 — Diff-Drive Trajectory Simulation

Set $W = 0.3$ m and simulate these configurations. Compare the result with the equivalent unicycle (v, ω) intuitively.

Straight Line: $v_L = v_R = 0.2$

$\omega = 0 \rightarrow$ robot drives straight. Equivalent to unicycle with $v = 0.2$, $\omega = 0$.

Spin in Place: $v_L = -v_R = 0.1$

$v = 0$, $\omega = 2v_L/W \rightarrow$ pure rotation. Only possible in diff-drive (not Ackermann)!

Arc: $v_L = 0.1$, $v_R = 0.3$

$v = 0.2$, $\omega = 0.2/W = 0.667$ rad/s $\rightarrow$ circular arc curving left.

Pivot on wheel: $v_L = 0$, $v_R = 0.2$

Robot pivots around the stationary left wheel. Tight turn.

Lab 10.5 — Basic Path-Following for Unicycle

This lab bridges kinematics with control — the foundation for AI-guided line following in Week 11.

Setup

Define a reference path (circle, sine curve, or straight line). At each time step, compute the heading error e_θ between the robot's current heading and the direction to the look-ahead point on the path.

Simple Proportional Controller

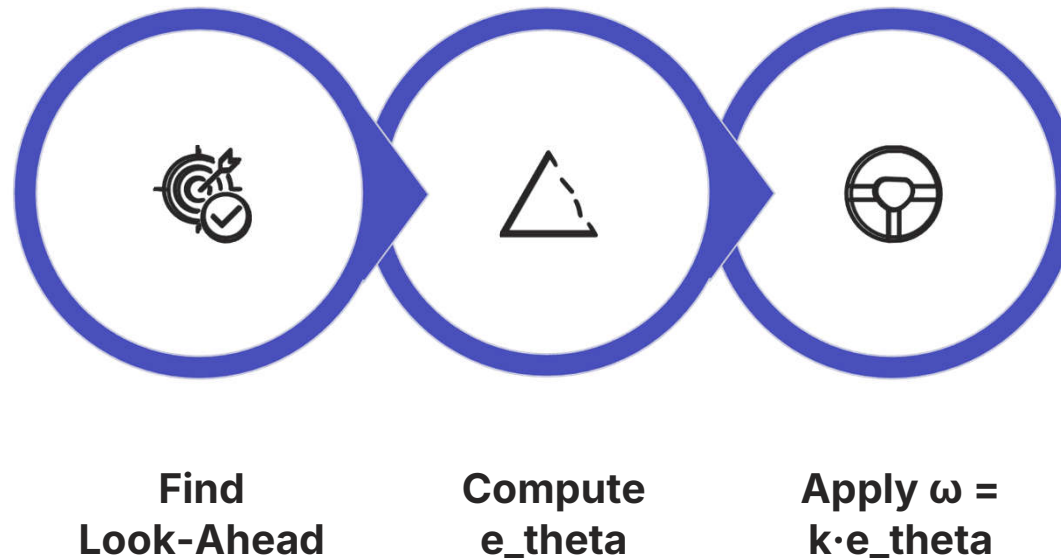
$$v \approx \text{const (e.g. 0.2)}$$

$$\omega = k e_\theta$$

Tune gain k and observe how quickly the robot converges to the path. Too large: oscillation. Too small: slow convergence.

 This look-ahead controller is conceptually identical to the line-following controller you will implement with a camera in Week 11.

Path-Following Controller — Look-Ahead Strategy



The look-ahead approach is robust and intuitive: the robot always steers toward a point slightly ahead on the desired path. The farther the look-ahead distance, the smoother (but less precise) the tracking. This same logic transfers directly to the visual line-following controller in Week 11.



Car-Like Robot & Ackermann Kinematics

Focus shifts to the Jet Racer: mapping SDK throttle and steering commands to physical velocity and steering angle, then simulating Ackermann trajectories.

Reviewing Ackermann Steering Geometry

The Core Principle

For a car turning a corner, all four wheels must rotate around a **single instantaneous center of rotation (ICR)**. If they don't, tires slip and wear out quickly.

Ackermann geometry ensures this by making the **inner front wheel turn more sharply** than the outer front wheel. In our kinematic model, we simplify this to a single **effective steering angle δ** .

□ At this level, you don't need to compute inner/outer wheel angles separately — the bicycle model with one δ is sufficient.

Key Parameters

L — Wheelbase

Distance from rear to front axle. Determines turning sensitivity.

δ — Steering angle

Effective front-wheel steering angle. Maps from SDK steering command.

v — Forward speed

Maps from SDK throttle command.

Lab 10.B1 — Writing `ackermann_step`

Implement a single Euler integration step for the Ackermann bicycle model:

```
def ackermann_step(x, y, th, v, delta, L, dt):  
    x_new = x + v * np.cos(th) * dt  
    y_new = y + v * np.sin(th) * dt  
    th_new = th + (v / L) * np.tan(delta) * dt  
    return x_new, y_new, th_new
```

Compare with Unicycle

The x and y equations are **identical** to the unicycle. Only $\dot{\theta}$ differs: instead of a free ω , it is $(v/L) \tan \delta$ — coupled to speed and steering geometry.

Typical Parameters (Jet Racer)

- $L \approx 0.17$ m (wheelbase)
- $\delta_{\max} \approx 0.5$ rad ($\sim 28^\circ$)
- $v_{\max} \approx 1.0$ m/s (throttle = 1.0)

Lab 10.B2 — Simulating Jet Racer Trajectories

Use `ackermann_step` in a simulation loop and explore these fundamental patterns. Compare each with the equivalent unicycle trajectory.

Straight Line

$\delta = 0 \rightarrow \omega = 0$, car drives straight. Same as unicycle with $\omega = 0$.

Constant Circle

$\delta = \text{const} \neq 0 \rightarrow \text{constant } \omega = (v/L) \tan \delta$
 $\rightarrow$ circular arc. Radius $R = L / \tan \delta$.

Slalom

$\delta = \delta_{\text{max}} \cdot \sin(t)$ $\rightarrow$ oscillating steering
 $\rightarrow$ S-shaped slalom trajectory.

 Notice: you cannot spin in place with Ackermann. The minimum turning radius is $R_{\text{min}} = L / \tan(\delta_{\text{max}})$.

Mapping Jet Racer SDK Commands to (v, δ)

SDK Interface

The Jet Racer Python SDK uses normalized commands:

- **steering** $s \in [-1, 1]$ (left = -1, right = +1)
- **throttle** $t \in [-1, 1]$ (reverse = -1, forward = +1)

Linear Mapping

$$v = t \times v_{\max}$$

$$\delta = s \times \delta_{\max}$$

Use these v and δ values directly in `ackermann_step`. This bridges the gap between software commands and physical motion — the same abstraction used on real hardware.

Lab 10.B3 — Wrapping SDK Commands into Ackermann Simulation

Write a unified wrapper that accepts SDK-level commands and returns the next simulated state. This creates a **hardware-in-the-loop compatible interface**:

```
V_MAX      = 1.0    # m/s
DELTA_MAX  = 0.5    # rad (~28 degrees)
L          = 0.17   # wheelbase in meters

def jet_racer_step(x, y, th, steering, throttle, dt):
    v      = throttle * V_MAX
    delta  = steering * DELTA_MAX
    return ackermann_step(x, y, th, v, delta, L, dt)
```

- ✓ Same function signature works for both simulation and real Jet Racer — just swap `ackermann_step` for actual SDK calls when on the real robot.



Preparing for Line Following & AI Control

Parts A and B built the kinematic tools. Part C connects them to the visual feedback loop that will drive both Turtlebot and Jet Racer in Week 11.

Reviewing the Error $\rightarrow (v, \omega)$ Control Law

In line following, a camera detects the line position and computes a lateral error e (pixels or normalized). The controller converts this into motion commands.

01

Detect Error

Camera detects line position. Compute error $e = (\text{line_x} - \text{image_center_x})$. Normalize to $[-1, 1]$.

03

Simulate Trajectory

Feed (v, ω) into `unicycle_step` or (steering, throttle) into `jet_racer_step` to preview the trajectory.

02

Compute Commands

Apply proportional controller: $\omega = k_e \cdot e$, $v = \text{const}$ (small, e.g. 0.15 m/s).

04

Deploy on Hardware

In Week 11, replace the simulation with actual camera + robot hardware. Same controller code — just real inputs.

Mapping Image Error → Steering for Jet Racer

Controller Design

Assume image error e is already normalized to $[-1, 1]$:

$$\text{steering} = k_s \cdot e$$

$$\text{throttle} = t_{\text{const}}$$

Through the SDK mapping $\rightarrow (v, \delta) \rightarrow \text{ackermann_step} \rightarrow \text{trajectory}$.

Tuning Guidelines

k_s too large

Oscillation / overcorrection — car weaves aggressively.

k_s too small

Slow correction — car drifts off line before recovering.

t_{const} too large

Too fast to correct steering in time — crashes into curves.

Maneuverability: Diff-Drive vs. Car-Like

Differential Drive (Turtlebot)

Spin in place — zero turning radius. Excellent in tight, cluttered environments. Ideal for indoor service robots. Slower top speed.



Car-like / Ackermann (Jet Racer)

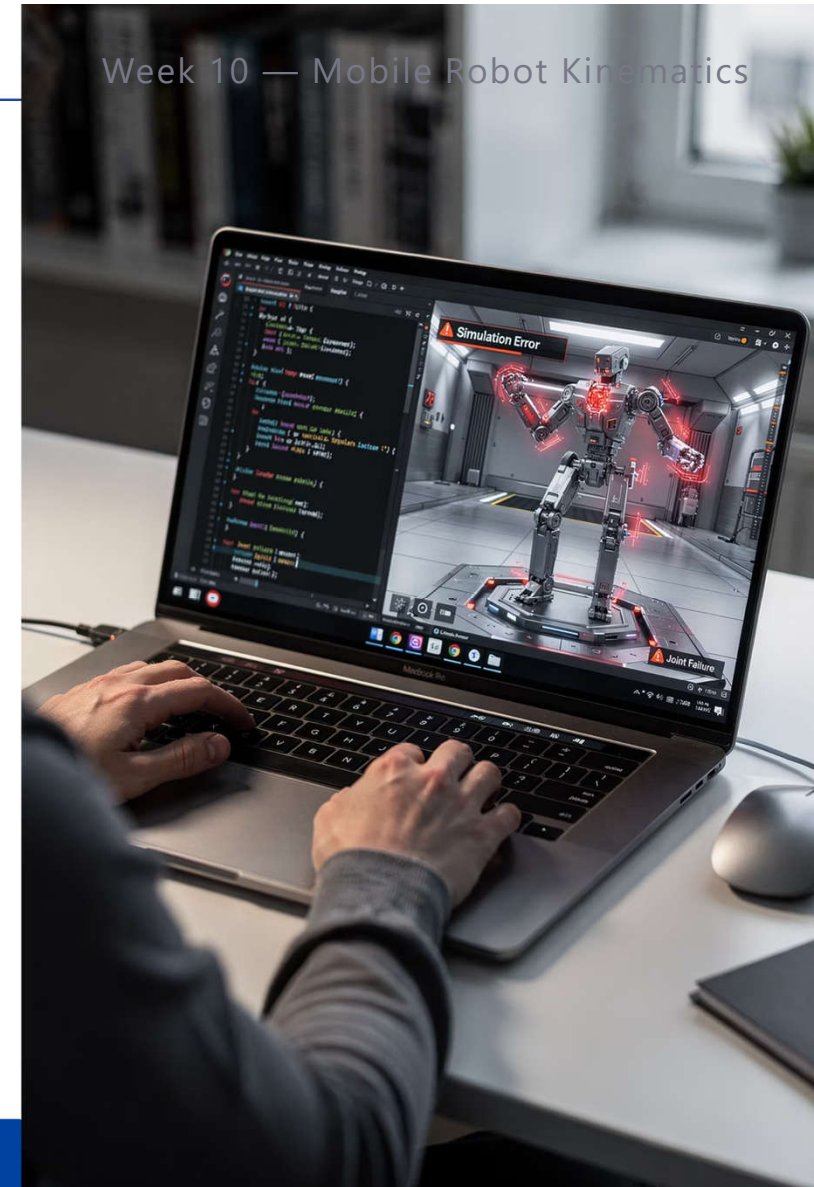
Finite turning radius — must plan ahead for curves. Better suited for **high-speed, structured environments** like corridors, roads, and tracks.

Using both platforms in RCV gives students direct, hands-on experience with both kinematic families and their practical trade-offs.

Common Errors in Mobile Robot Simulation

These are the most frequent mistakes students make. Catching them early saves significant debugging time.

Week 10 — Mobile Robot Kinematics



Top Simulation Bugs & How to Fix Them

Degrees vs. Radians

θ and δ must be in **radians** in all kinematic equations. If you initialize θ in degrees (e.g. 90 instead of $\pi/2$), trajectories will appear wildly incorrect. Always use `np.deg2rad()` for conversion.

Time Step dt Too Large

Euler integration accumulates error proportional to dt . Use **$dt \leq 0.05$ s** for smooth trajectories. Compare with the analytic circle formula to verify accuracy.

Wrong Axis Convention

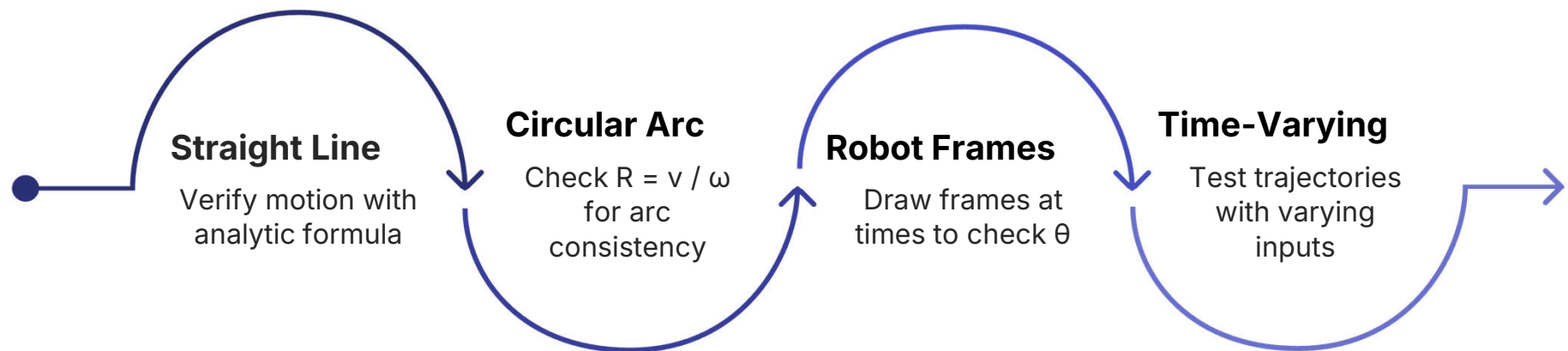
In standard robot convention: **x points forward, y points left, $\theta = 0$ faces +x**. Mixing with image coordinates (y pointing down) causes mirrored or rotated trajectories.

Forgetting `axis('equal')`

Without equal axis scaling, a circle looks like an ellipse. Always call `plt.axis('equal')` when visualizing robot trajectories.

How to Debug Trajectories Systematically

Always start from the simplest case and build up complexity. This isolates bugs to a specific component.



If step 1 fails, your integration or angle convention is wrong. If step 2 fails, check the ω formula or dt . If heading direction looks wrong in step 3, check the axis convention. Only proceed to step 4 after all basic cases pass.

Connection to Visual Odometry & SLAM



Kinematics as a Motion Prior

The unicycle/diff-drive/Ackermann models are not just for trajectory simulation — they serve as the **motion model** in Visual Odometry and SLAM algorithms. In VO/SLAM, the kinematic model **predicts** where the robot should be between two camera frames. This prediction is then **corrected** using visual feature matching.

- Week 12 will revisit these models directly when discussing VO and map building.

Summary: Unicycle & Differential Drive



Unicycle Model

Abstract model: state (x, y, θ) , inputs (v, ω) .
Foundation for mobile robot planning and control. Describes nonholonomic constraint naturally.



Differential Drive

Physical realization of unicycle via wheel speeds (v_L, v_R) . Bidirectional mapping: FK and IK both simple linear formulas.



Two-Layer Architecture

Design controllers at (v, ω) level; driver handles wheel mapping. Turtlebot uses this architecture natively.

Summary: Ackermann & Jet Racer



Ackermann Kinematic Model

State (x, y, θ) , inputs (v, δ) . Angular rate $\omega = (v/L) \tan \delta$ — coupled to speed. Finite minimum turning radius.



Jet Racer SDK Mapping

steering/throttle $\in [-1, 1] \rightarrow (v, \delta)$. Use same `ackermann_step` for simulation and real deployment.



More Complex Control

v and ω are **not independent** — you cannot set ω without considering v . Requires more careful controller design than diff-drive.

Formula Reference Card

All the essential equations from Week 10 in one place.

Unicycle

$$\dot{x} = v \cos \theta$$

$$\dot{y} = v \sin \theta$$

$$\dot{\theta} = \omega$$

Diff-Drive Mapping

$$v = \frac{v_R + v_L}{2}$$

$$\omega = \frac{v_R - v_L}{W}$$

$$v_{R,L} = v \pm \omega \frac{W}{2}$$

Ackermann

$$\dot{x} = v \cos \theta$$

$$\dot{y} = v \sin \theta$$

$$\dot{\theta} = \frac{v}{L} \tan \delta$$

Week 10 Complete — Looking Ahead to Week 11

You now have the full kinematic toolkit for both arm robots (Weeks 1–9) and mobile robots (Week 10). Week 11 will put these models to work using real camera data and AI-based line detection to drive Turtlebot and Jet Racer along a track.



What's Coming in Weeks 11–13

Week 11

Line following with AI/camera: error $\rightarrow (v, \omega) \rightarrow$ unicycle/Ackermann on real hardware

Week 13

Obstacle avoidance: combine vision + kinematics + reactive planning

Week 12

Visual Odometry: use kinematic motion models as priors between camera frames

✓ The final project may combine arm manipulation (Weeks 1–9) and mobile navigation (Weeks 10–13) in a single integrated demo.

Q&A — Mobile Robot Kinematics

Let's Discuss!

Think about real-world examples: self-driving cars (Ackermann), warehouse robots like Amazon Kiva (diff-drive), delivery robots on sidewalks...

Why can't a car parallel-park in a single motion but a Turtlebot can?

What happens to the Ackermann model if we set $\delta_{\max}$ very large?

How would you modify the unicycle model if wheels slip on a wet surface?

